~/majibill/docs/02-tech-stack-adrs.md

$ cat 02-tech-stack-adrs.md

MAJIBILL PLATFORM · STACK

Tech Stack & Architecture Decision Records

A deliberate multistack: each language and tool only where it is strongest, with the trade-offs and the triggers for change written down.

```
tenant_id = current_setting('app.tenant_id')::uuid
-- row level security: ON
-- every write is audited, every payment is idempotent
```

DOC	VERSION	DATE
MB-02	1.0 draft	08 Oct 2026

STATUS	AUDIENCE	CLASS
For review	Engineering, technical reviewers, future hires	Confidential

Prepared by 404HubSpot · Nakuru, Kenya · https://404-hub-spot.vercel.app
Working codename: MajiBill (Swahili: maji = water). Figures marked 'assumption' must be validated before commitment.

// Contents

1. How we choose 3

2. Stack at a glance 4

3. Why TypeScript and not ASP.NET Core? 5

4. Architecture Decision Records 6

// 1. How we choose

The team can write in several languages and works with agentic workflows, so syntax is not the constraint. The constraints are **operability, correctness of money flows, hiring, review capacity, and cost in the Kenyan market**. Every choice below is scored against those, not against taste.

Criterion	Question we ask
Correctness	Does it make wrong-money bugs harder to write? (types, transactions, constraints)
Operability	Can two engineers on call run it at 2 a.m.? Is there a managed option?
Review capacity	Can humans actually review the agent-written code in it? Fewer languages in the money path = better review.
Ecosystem for Kenya	M-Pesa/Daraja, SMS gateways, local payment aggregators: libraries, examples, community.
Portability	Can it run in a client's environment (sovereign cell) without a proprietary cloud service?
Cost	Licence, hosting, and talent cost at startup scale and at Nairobi scale.

1.1 The multistack rule

> DECISION **One language per layer; a new language needs an ADR, an owner and two on-call engineers**
> Multistack works when boundaries are contracts (OpenAPI, JSON Schema, SQL) and each stack has an owner. It fails when every engineer picks a favourite. The rule keeps the freedom you want without the sprawl.

// 2. Stack at a glance

Layer	Choice	Considered	Why this one
Core API + workers	TypeScript, NestJS, Zod, Drizzle (SQL-first)	ASP.NET Core 8, Go, Python/Django	Biggest surface area; shares types/contracts with web; fastest iteration with agents; strong enough typing for domain code
Payment ingress	Go	Node, Rust	Small static binary, predictable latency, trivial to run beside anything
Analytics / ETL	Python (pandas, SQL, scikit-learn)	TypeScript, R	Best ecosystem for anomaly detection, forecasting, data migration
Web apps	Next.js 16, React 19, Tailwind 4, shadcn/ui	Vite SPA, Angular	Matches the `sales-crm` starter and team conventions; SSR for portal; component ecosystem
Reader app	Flutter (Dart), Drift (SQLite)	React Native (Expo)	Strong offline story, consistent performance on low-end Android, one codebase for Android and iOS
Database	PostgreSQL 16+	MySQL, SQL Server, MongoDB	RLS, partitioning, JSONB, PITR, mature managed offerings, open licence
Queue / cache	Redis + BullMQ	RabbitMQ, SQS, Kafka	Operationally light; enough for volumes here; Kafka is premature
Identity	In-house module on proven libraries (Argon2id, sessions, TOTP)	Keycloak, Auth0, Clerk	Data residency, cost per MAU, tenant-aware RBAC; Keycloak added per tenant if SSO is required
Object storage	S3-compatible	Local disk	Durable, cheap, supports object lock for audit anchors
IaC	Terraform / OpenTofu	Pulumi, CDK	Widely known, portable across providers
Compute	Containers on ECS Fargate (or equivalent); Helm later	Kubernetes day one	Fewer moving parts for a small team; portable images
CI/CD	GitHub Actions, Trivy, Dependabot, CodeRabbit	GitLab, Jenkins	Already in the baseline workflow
Observability	OpenTelemetry, Prometheus/Grafana (or Grafana Cloud), Loki, Sentry	Datadog	Open standards, controllable cost
API style	REST + OpenAPI 3.1, webhooks	GraphQL, gRPC	Partner/mobile friendly, easy codegen, cacheable

// 3. Why TypeScript and not ASP.NET Core?

ASP.NET Core is a strong choice for billing: mature, fast, excellent tooling and typing. This is a real alternative, not a strawman. The recommendation for the core is TypeScript because of the team and delivery model, not because .NET is weak.

Factor	NestJS / TypeScript	ASP.NET Core 8
Shared types with web	**Native** (same language, shared packages)	Via OpenAPI codegen
Agentic workflow leverage	Very high (huge corpus, fast feedback loops)	High
Raw performance	Adequate for this load (about 100 bills/s, low payment TPS)	Higher ceiling
Typing for money logic	Good with strict TS + Zod + branded types	**Excellent** (decimal type, records)
Ecosystem for Daraja/SMS	Many examples and SDKs	Fewer, but simple HTTP
Hiring in Kenya	Broad	Moderate
Ops / container size	Larger images, Node runtime	Smaller, faster start

> NOTE **Reversible by design**
> Because modules talk via public interfaces and the database is the contract, the core could be re-platformed module by module if evidence demands it. Money is stored as integer minor units or fixed-scale `numeric`, never floating point, in either language.

3.1 Repository layout (monorepo, polyglot)

repo layout

```
majibill/
  apps/
    web/              # Next.js: staff app + customer portal (route groups)
    console/          # Next.js: platform console for 404HubSpot + partners
    reader/           # Flutter: offline meter reading app
  services/
    core-api/         # NestJS modular monolith (modules/, shared/, migrations/)
    workers/          # same domain modules, queue consumers
    payment-ingress/  # Go: Daraja callbacks -> inbox table
    analytics/        # Python: anomaly, NRW, forecasting, ETL
  packages/
    contracts/        # OpenAPI + JSON Schema, generated clients (TS, Dart, Go, Python)
    ui/               # shared React components, design tokens
    config/           # eslint, tsconfig, prettier, shared lint rules
  infra/
    terraform/        # modules: cell, network, db, observability
    docker/           # base images, compose for sovereign installs
  docs/               # decisions/, runbooks/, specs/ (baseline project-kit layout)
  AGENTS.md           # agent rules and reading order
```

// 4. Architecture Decision Records

ADR-001 Modular monolith core, three satellites	ACCEPTED
Context	Small team, one client, regulated money flow, need for sovereign installs.
Decision	One deployable core with enforced module boundaries; split out only payment ingress, workers, analytics.
Consequences	Fast start, strong consistency, simple on-prem. Requires boundary discipline and CI import-graph checks.
Alternatives	Full microservices; unstructured monolith.

ADR-002 Multitenancy: pooled RLS by default, silo on demand	ACCEPTED
Context	Many tenants of very different sizes; some will demand isolation; one may want to buy.
Decision	`tenant_id` on every tenant table with RLS; same artifact deployable as dedicated cell.
Consequences	Cheap onboarding; strong isolation path. RLS tests are mandatory; noisy-neighbour limits needed.
Alternatives	Schema-per-tenant; DB-per-tenant for all.

ADR-003 Ledger-based accounting	ACCEPTED
Context	Balances must be explainable, auditable, correctable.
Decision	Append-only ledger entries; balances derived; reversals instead of edits; allocation rules explicit.
Consequences	Auditors can replay history; slightly more write logic; reports need read models.
Alternatives	Mutable balance column.

ADR-004 Payments via Go ingress + inbox pattern	ACCEPTED
Context	Daraja callbacks are bursty and must be acked quickly and exactly-once in effect.
Decision	Go service writes callbacks to an `inbox` table with a unique key, then returns 200; workers process.
Consequences	Decouples receipt from processing; replays are harmless. One more deployable to run.
Alternatives	Process inside core API.

ADR-005 Each tenant has its own M-Pesa shortcode	PROPOSED
Context	Platform should not hold customer funds; settlement should go to the utility.
Decision	Per-tenant Paybill/Till configured in the registry; credentials in a secrets manager.
Consequences	Cleaner compliance position; onboarding needs shortcode setup per tenant. Confirm with Safaricom and counsel.
Alternatives	Shared aggregator shortcode.

ADR-006 Offline-first reader app with server-arbitrated sync	ACCEPTED
Context	Field connectivity is unreliable.
Decision	Flutter + SQLite (Drift); client UUIDs; delta sync; server validates and flags; conflicts surfaced to a supervisor.
Consequences	Works for a full route-day offline; sync logic needs thorough tests.
Alternatives	Online-only app; React Native.

ADR-007 In-house identity module	ACCEPTED
Context	Tenant-aware RBAC, phone-OTP customers, data residency, MAU-based pricing risk.
Decision	Argon2id, rotating sessions, TOTP for staff, optional SSO adapter (OIDC/SAML) per tenant.
Consequences	Full control and lower cost; we own the security surface (mitigated by libraries, tests, pen-tests).
Alternatives	Auth0, Clerk, Keycloak everywhere.

ADR-008 Postgres-first events (transactional outbox)	ACCEPTED
Context	Need reliable events without running a broker on day one.
Decision	Outbox table in the same transaction; relay to Redis queues; broker introduced when consumers multiply.
Consequences	One fewer system; careful relay design.
Alternatives	Kafka/NATS from day one.

ADR-009 Rust reserved, not used in v1	ACCEPTED
Context	Rust is attractive for a shared deterministic tariff engine, but adds a third build chain to the money path.
Decision	Do not use Rust until profiling or cross-platform correctness (server + offline estimates) justifies it.
Consequences	Less to review; revisit at month 9 with real data.
Alternatives	Rust tariff engine now.

ADR-010 SQL-first migrations, expand/contract	ACCEPTED
Context	RLS, partitioning, triggers and grants are SQL; ORMs hide them.
Decision	Plain SQL migrations (reviewed), ORM only for query building; backwards-compatible deploys.
Consequences	Safer zero-downtime releases; requires migration review discipline.
Alternatives	ORM-managed migrations.

4.1 When we revisit

Signal	Revisit
p95 bill-run time exceeds target after tuning	Extract billing calculation into a dedicated service, evaluate Go/Rust
Reporting load degrades OLTP	Columnar analytics store and CDC
Tenants demand SSO	Add Keycloak adapter per tenant
Event consumers exceed three	Introduce NATS JetStream / Kafka
Single Postgres write rate becomes limiting	Further cell splits first; sharding last